Missions backend et frontend de l’équipe

Dabber Khdma · Classe SSA 9

Ce document regroupe les responsabilités des six membres pour les six microservices métier. Le socle commun couvre les comptes et l’authentification. Chaque moteur IA est appelé uniquement par son backend propriétaire.

Mission de Rayen — Catalogue de projets

Classe SSA 9 · microservice métier 1 sur 6. Cette fiche décrit le travail à réaliser.

Backend

Créer, modifier, publier, rechercher et annuler les projets. Contrôler le propriétaire, les états et les budgets. Assurer la réservation atomique d'une attribution, avec clé idempotente et verrou partagé avec l'annulation. Émettre les événements depuis une outbox.

Données propriétaires : base catalog. Le backend valide les entrées, vérifie l'identité et les droits sur chaque ressource, conserve ses données et renvoie des erreurs compréhensibles. Les références vers les autres services sont des identifiants externes.

Communication synchrone : Proposal appelle POST /internal/projects/{id}/award-reservations avec une identité de service et une clé stable.

Communication asynchrone : project.published.v1. Les consommateurs utilisent des traitements idempotents et des reprises bornées.

Frontend

Créer les écrans de publication, modification, recherche et filtres. Afficher les détails, les compétences demandées, les états du projet et les erreurs de validation.

Les pages appartiennent à l'application Next.js commune et utilisent les API du backend via la Gateway. Elles doivent gérer chargement, données absentes, accès refusé et erreurs, ainsi que la navigation clavier et mobile.

Écrans et contrats à relier

- POST /projects : Créer un brouillon — 201.
- GET /projects : Rechercher les projets ouverts — 200.
- GET /projects/{id} : Lire un projet visible — 200.
- POST /projects/{id}/publish : Publier son projet — 200.
- POST /projects/{id}/cancel : Annuler avant réservation — 200.

Les contrats complets, entrées, réponses et erreurs se trouvent dans Documentation/Contrats/project-catalog.openapi.json.

Ordre de réalisation proposé

1. Définir les règles métier et les modèles de données.
2. Implémenter les API, validations, autorisations et tests backend.
3. Réaliser les pages et le client API frontend avec les états d'erreur.
4. Intégrer les échanges avec les services concernés et les événements.
5. Vérifier le parcours complet, les accès refusés et les reprises.

Critères de fin

- Un brouillon reste privé; deux demandes concurrentes ne peuvent réserver le même projet.
- Les données appartiennent au service et les accès sont vérifiés côté backend.
- Les écrans et contrats décrivent les mêmes comportements et états.
- Une fonctionnalité IA future est appelée par ce backend, jamais directement par le frontend.
- Les tests métier et d'intégration sont réalisés avec l'implémentation.

Mission de Rana — Propositions et sélection

Classe SSA 9 · microservice métier 2 sur 6. Cette fiche décrit le travail à réaliser.

Backend

Gérer les propositions, leurs versions, le retrait, la présélection et l'attribution. Vérifier le projet et son propriétaire auprès du catalogue; réserver avant de créer la sélection. Publier proposal.accepted et suivre la création du contrat.

Données propriétaires : base proposal. Le backend valide les entrées, vérifie l'identité et les droits sur chaque ressource, conserve ses données et renvoie des erreurs compréhensibles. Les références vers les autres services sont des identifiants externes.

Communication synchrone : Vérifie le projet et son propriétaire dans Catalog; réserve l'attribution avant le commit local de Selection et OutboxEvent.

Communication asynchrone : proposal.accepted.v1. Les consommateurs utilisent des traitements idempotents et des reprises bornées.

Frontend

Créer les formulaires de candidature, la liste des propositions, leur comparaison et la sélection. Après un retour 202, afficher l'attente et interroger le statut de la sélection.

Les pages appartiennent à l'application Next.js commune et utilisent les API du backend via la Gateway. Elles doivent gérer chargement, données absentes, accès refusé et erreurs, ainsi que la navigation clavier et mobile.

Écrans et contrats à relier

- POST /projects/{id}/proposals : Déposer une proposition — 201.
- GET /projects/{id}/proposals : Consulter les propositions autorisées — 200.
- POST /proposals/{id}/shortlist : Présélectionner — 200.
- POST /proposals/{id}/withdraw : Retirer sa proposition — 200.
- POST /proposals/{id}/accept : Lancer une attribution idempotente — 202.
- GET /selections/{id} : Suivre la création du contrat — 200.

Les contrats complets, entrées, réponses et erreurs se trouvent dans Documentation/Contrats/proposal-selection.openapi.json.

Ordre de réalisation proposé

6. Définir les règles métier et les modèles de données.
7. Implémenter les API, validations, autorisations et tests backend.
8. Réaliser les pages et le client API frontend avec les états d'erreur.
9. Intégrer les échanges avec les services concernés et les événements.
10. Vérifier le parcours complet, les accès refusés et les reprises.

Critères de fin

- Une seule proposition active par freelancer et projet; une seule attribution; une clé répétée ne crée pas de doublon.
- Les données appartiennent au service et les accès sont vérifiés côté backend.
- Les écrans et contrats décrivent les mêmes comportements et états.
- Une fonctionnalité IA future est appelée par ce backend, jamais directement par le frontend.
- Les tests métier et d'intégration sont réalisés avec l'implémentation.

Mission de Yassin — Contrats et jalons

Classe SSA 9 · microservice métier 3 sur 6. Cette fiche décrit le travail à réaliser.

Backend

Créer le contrat de manière idempotente après sélection. Gérer jalons, livrables, demandes de révision, approbations et litiges; contrôler les participants et transitions. Publier les événements de contrat et jalon.

Données propriétaires : base engagement. Le backend valide les entrées, vérifie l'identité et les droits sur chaque ressource, conserve ses données et renvoie des erreurs compréhensibles. Les références vers les autres services sont des identifiants externes.

Communication synchrone : Communication appelle GET /contracts/{id} pour vérifier les participants; Engagement lit le stockage objet pour valider les métadonnées d'un livrable.

Communication asynchrone : contract.created.v1 / milestone.approved.v1. Les consommateurs utilisent des traitements idempotents et des reprises bornées.

Frontend

Créer le tableau des contrats, la vue des jalons, le dépôt de livrables, la demande de révision et l'approbation. Afficher clairement les états et les erreurs.

Les pages appartiennent à l'application Next.js commune et utilisent les API du backend via la Gateway. Elles doivent gérer chargement, données absentes, accès refusé et erreurs, ainsi que la navigation clavier et mobile.

Écrans et contrats à relier

- GET /contracts/{id} : Lire son contrat — 200.
- POST /contracts/{id}/milestones : Créer un jalon — 201.
- POST /milestones/{id}/submit : Soumettre les métadonnées du livrable — 200.
- POST /milestones/{id}/approve : Approuver un livrable — 200.
- POST /milestones/{id}/request-revision : Demander une révision — 200.
- POST /contracts/{id}/disputes : Ouvrir un litige — 201.
- POST /milestones/{id}/start : Démarrer un jalon planifié — 200.

Les contrats complets, entrées, réponses et erreurs se trouvent dans Documentation/Contrats/engagement.openapi.json.

Ordre de réalisation proposé

11. Définir les règles métier et les modèles de données.
12. Implémenter les API, validations, autorisations et tests backend.
13. Réaliser les pages et le client API frontend avec les états d'erreur.
14. Intégrer les échanges avec les services concernés et les événements.
15. Vérifier le parcours complet, les accès refusés et les reprises.

Critères de fin

- Un tiers ne peut accéder au contrat; un litige ouvert bloque la clôture; un événement rejoué ne crée pas deux contrats.
- Les données appartiennent au service et les accès sont vérifiés côté backend.
- Les écrans et contrats décrivent les mêmes comportements et états.
- Une fonctionnalité IA future est appelée par ce backend, jamais directement par le frontend.
- Les tests métier et d'intégration sont réalisés avec l'implémentation.

Mission de Farah — Communication

Classe SSA 9 · microservice métier 4 sur 6. Cette fiche décrit le travail à réaliser.

Backend

Gérer conversations, participants, messages et pièces jointes. Vérifier les participants auprès du service Contrats et à chaque accès. Persister message et outbox avant de publier message.sent.

Données propriétaires : base communication. Le backend valide les entrées, vérifie l'identité et les droits sur chaque ressource, conserve ses données et renvoie des erreurs compréhensibles. Les références vers les autres services sont des identifiants externes.

Communication synchrone : Lit les participants du contrat par GET /contracts/{id} lors de l'ouverture; contrôle leur appartenance à chaque lecture/écriture.

Communication asynchrone : message.sent.v1. Les consommateurs utilisent des traitements idempotents et des reprises bornées.

Frontend

Créer la liste des conversations, l'historique paginé, le formulaire de message, les pièces jointes et les états de lecture. Prévoir chargement, absence de messages et erreurs.

Les pages appartiennent à l'application Next.js commune et utilisent les API du backend via la Gateway. Elles doivent gérer chargement, données absentes, accès refusé et erreurs, ainsi que la navigation clavier et mobile.

Écrans et contrats à relier

- GET /conversations : Lister mes conversations — 200.
- GET /conversations/{id}/messages : Historique paginé — 200.
- POST /conversations/{id}/messages : Envoyer un message — 201.
- POST /messages/{id}/read : Marquer la lecture — 200.

Les contrats complets, entrées, réponses et erreurs se trouvent dans Documentation/Contrats/communication.openapi.json.

Ordre de réalisation proposé

16. Définir les règles métier et les modèles de données.
17. Implémenter les API, validations, autorisations et tests backend.
18. Réaliser les pages et le client API frontend avec les états d'erreur.
19. Intégrer les échanges avec les services concernés et les événements.
20. Vérifier le parcours complet, les accès refusés et les reprises.

Critères de fin

- Un tiers ne peut lire ou envoyer de message; le texte du message ne circule pas dans les événements.
- Les données appartiennent au service et les accès sont vérifiés côté backend.
- Les écrans et contrats décrivent les mêmes comportements et états.
- Une fonctionnalité IA future est appelée par ce backend, jamais directement par le frontend.
- Les tests métier et d'intégration sont réalisés avec l'implémentation.

Mission de Oussama — Notifications

Classe SSA 9 · microservice métier 5 sur 6. Cette fiche décrit le travail à réaliser.

Backend

Consommer les événements dans des files dédiées; gérer boîte, préférences, tentatives de livraison, déduplication, reprises et DLQ. Obtenir les coordonnées par le socle comptes avec une identité de service.

Données propriétaires : base notification. Le backend valide les entrées, vérifie l'identité et les droits sur chaque ressource, conserve ses données et renvoie des erreurs compréhensibles. Les références vers les autres services sont des identifiants externes.

Communication synchrone : Le worker courriel récupère les coordonnées autorisées via une API interne limitée du socle Utilisateurs, puis contacte le fournisseur de messagerie.

Communication asynchrone : notification.email.requested.v1. Les consommateurs utilisent des traitements idempotents et des reprises bornées.

Frontend

Créer le centre de notifications, le compteur non lu, les actions de lecture et les préférences. Les liens ouvrent uniquement des ressources accessibles à l'utilisateur.

Les pages appartiennent à l'application Next.js commune et utilisent les API du backend via la Gateway. Elles doivent gérer chargement, données absentes, accès refusé et erreurs, ainsi que la navigation clavier et mobile.

Écrans et contrats à relier

- GET /notifications : Lire ma boîte de réception — 200.
- POST /notifications/{id}/read : Marquer comme lue — 200.
- GET /notification-preferences : Lire mes préférences — 200.
- PATCH /notification-preferences : Modifier mes préférences — 200.

Les contrats complets, entrées, réponses et erreurs se trouvent dans Documentation/Contrats/notification.openapi.json.

Ordre de réalisation proposé

21. Définir les règles métier et les modèles de données.
22. Implémenter les API, validations, autorisations et tests backend.
23. Réaliser les pages et le client API frontend avec les états d'erreur.
24. Intégrer les échanges avec les services concernés et les événements.
25. Vérifier le parcours complet, les accès refusés et les reprises.

Critères de fin

- Un événement rejoué ne crée pas de doublon; les préférences sont respectées; les échecs finaux sont traçables.
- Les données appartiennent au service et les accès sont vérifiés côté backend.
- Les écrans et contrats décrivent les mêmes comportements et états.
- Une fonctionnalité IA future est appelée par ce backend, jamais directement par le frontend.
- Les tests métier et d'intégration sont réalisés avec l'implémentation.

Mission d’Ayoub — Profil professionnel, CV et matching

Classe SSA 9 · microservice métier 6 sur 6. Le socle authentification/comptes est commun et hors décompte. Cette fiche décrit le travail à réaliser.

Backend

Ayoub réalise le backend métier Profil professionnel/CV/Matching, identifié par le dossier ai-matching. Les routes publiques sont servies par Spring Boot et passent par la Gateway. Le moteur Python/FastAPI est privé et séparé de ces routes.

La couche contrôleur reçoit et valide les DTO; la couche applicative orchestre les cas d’usage; le domaine impose propriété, confirmation et versions; les repositories conservent les données PostgreSQL. Un AIClient avec adaptateur HTTP est la seule dépendance vers le moteur privé. Un faux adaptateur déterministe doit permettre les tests sans modèle externe.

Gestion du profil : lire et modifier son titre, biographie, compétences, expériences, formations, langues, disponibilité et portfolio; vérifier les tailles et URLs; gérer visibilité privée/publique et expectedProfileVersion. GET /profiles/{id} retourne uniquement un profil autorisé ou public, sans CV ni contact privé du compte.

Gestion du CV : contrôler PDF/DOCX, taille, quarantaine et propriétaire; enregistrer objet et métadonnées; créer une tâche durable; exposer statut et suggestions; appliquer seulement les champs confirmés; permettre remplacement et suppression sans résurrection par un worker tardif.

Matching : lire le profil confirmé localement, appeler Catalog pour les offres, créer le job backend et faire appeler /v1/score par son worker. Valider les résultats, versionner profil/offre/modèle, persister les raisons et exposer les états et résultats par API métier. Un timeout ou 503 du moteur produit reprise puis échec visible, sans bloquer la recherche.

Fiabilité : transaction métier + outbox, inbox/déduplication, délais et retries bornés, DLQ et journaux sans CV. Le backend contrôle l’état du compte et la propriété à chaque accès. Il n’expose aucun rôle, contact vérifié ou credential IA dans un formulaire de profil.

Livrables backend : contrat OpenAPI public, contrat IA interne, modèles et migrations, contrôleurs/services/repositories, AIClient, workers, configuration privée, scénarios d’autorisation et de concurrence, tests d’intégration et guide de reprise. Ces éléments sont des travaux à implémenter; le pack fournit leur conception.

Frontend

Ayoub réalise les pages du domaine dans le frontend Next.js partagé. Les appels utilisent un client API typé dirigé vers la Gateway et le backend métier. Aucun fetch vers FastAPI, aucune URL du moteur IA et aucun secret de service ne doivent apparaître dans le navigateur.

Éditeur de profil : sections titre et biographie, compétences et niveaux déclarés, expériences, formations, langues, disponibilité et portfolio. Permettre ajout, modification et suppression des éléments, afficher les validations et la version courante, sauvegarder par PATCH /profiles/me et préserver les saisies en cas d’erreur.

Import du CV : sélection d’un fichier, format et taille indiqués, message de confidentialité, progression de téléversement distincte du traitement. Après 202, suivre Location via le backend avec des requêtes bornées; afficher contrôle, attente, extraction, succès et erreur. Le navigateur ne sait pas quel moteur calcule en arrière-plan.

Relecture : présenter les suggestions champ par champ avec source et confiance indicative, permettre correction, sélection et exclusion. Rien n’est confirmé par défaut en silence. Le bouton de confirmation transmet resumeId, extractionVersion, expectedProfileVersion et confirmedFields au backend métier.

Profil public : fournir une prévisualisation et une commande explicite de visibilité. Montrer les informations professionnelles autorisées, sans fichier CV, adresse privée, rôles ou flags de sécurité. La saisie manuelle reste disponible si le CV échoue.

Recommandations : lancer POST /matches, afficher l'attente puis les offres classées, compétences communes/manquantes et versions. Permettre filtre vers une offre, accès au détail et candidature via le module de Rana. Signaler résultat obsolète et offrir un recalcul. La recherche Catalogue de Rayen reste utilisable sans IA.

Erreurs : 401 déclenche le parcours de connexion; 403/404 masque une ressource non autorisée; 409 propose de recharger et fusionner au lieu d'écraser; 413/415 explique le fichier refusé; 422 propose correction ou saisie manuelle; 429/503 fournit une reprise compréhensible. Les composants doivent être accessibles au clavier et sur mobile.

Livrables frontend : pages, formulaires et composants réutilisables, client API typé, gestion des jobs et états d'erreur, prévisualisation et recommandations, mocks cohérents avec les contrats et tests du parcours CV → profil confirmé → offres. Le frontend n'exécute aucune décision d'autorisation ou de scoring à la place du backend.

Écrans et contrats à relier

Écran	API du backend métier	Résultat attendu
Mon profil	GET /profiles/me	Charger les sections privées et leur version
Modifier mon profil	PATCH /profiles/me	Valider puis sauvegarder les champs modifiés
Importer mon CV	POST /resumes	202 puis suivi de Location
Suivi du CV	GET /resumes/{id}	Montrer le traitement backend et ses erreurs
Relire les suggestions	GET /resumes/{id}/prefill	Proposer des champs modifiables et sélectionnables
Confirmer	POST /profiles/me/resume-fill	Persister uniquement les champs acceptés
Supprimer le CV	DELETE /resumes/{id}	Suppression traçable sans supprimer implicitement le profil
Profil public	GET /profiles/{id}	Afficher seulement les informations autorisées
Demander des offres	POST /matches	Créer le job backend et retourner 202
Suivre les résultats	GET /matching-jobs/{id} et /results	Présenter recommandations et explications

Le moteur privé expose seulement /v1/extract et /v1/score à l'adaptateur du backend. Ces deux routes ne figurent pas dans le client frontend.

Ordre de réalisation proposé

26. Modèle de profil et CRUD manuel, droits et version optimiste.
27. Formulaire frontend et visibilité, sans dépendance IA.
28. Import privé, stockage et suivi de tâche avec faux adaptateur IA.
29. Liaison backend → moteur IA, extraction et relecture des suggestions.
30. Confirmation atomique, suppression et tests de reprises.
31. Lecture des offres, scoring interne et page de recommandations.
32. Intégration avec Catalogue, Propositions et Notifications, puis validation de bout en bout.

Critères de fin

- Aucun appel frontend → moteur IA, aucune clé IA dans le bundle.
- Un utilisateur ne lit ni CV, ni profil privé, ni job d'un tiers.
- Le profil se modifie manuellement sans IA; le CV n'écrit rien sans confirmation.

- Un conflit 409 n’écrase pas le formulaire ou la version courante.
- Une suppression ou nouvelle version neutralise les résultats tardifs.
- Le matching utilise les champs confirmés et indique les versions et les raisons.
- Une panne IA n’interrompt pas profil, catalogue ou candidature.
- Les API, écrans, diagrammes et tests décrivent le même propriétaire des données.